

Table of contents

Chapter 2. Streaming audiobooks to your phone with Audiobookshelf	1
Prerequisites	2
Part A: Rip and copy the audio	2
Step 1: Inspect what's actually on the disc	2
Step 2: Use the one-file-per-lesson folder	3
Step 3: Get the audio onto the Pi	3
Part B: Run the server	4
Step 4: Run Audiobookshelf on the Pi	4
Part C: Connect and use it	5
Step 5: Initial setup in the browser	5
Step 6: Connect the iPhone	6
Step 7: Daily use	6
Maintenance	7
Troubleshooting	7
Recap	7

Chapter 2. Streaming audiobooks to your phone with Audiobookshelf

The payoff of this chapter: a collection of spoken audio (audiobooks, podcasts, language courses, anything) copied once and served from the Raspberry Pi you set up in Chapter 1, streaming to your phone from anywhere over your tailnet, with proper audiobook-style resume tracking (pause mid-sentence today, resume at the same second tomorrow).

i The concept is general; the example is mine

This part shows how to serve **any** spoken audio from your Pi. The running example I use throughout is my own use case: a language-learning audio course I ripped from CD, because what I actually wanted was to study on my lunch break with just the book and none of the discs. Wherever you see the course, picture your own audiobooks; the steps are the same.

In [Chapter 1](#) you built the platform: a Pi named **homelab** running Docker and on your tailnet, reachable by name from your laptop and iPhone. Now we put it to work with **Audiobookshelf**, a self-hosted media server purpose-built for spoken audio. It runs in a Docker container, scans a folder of MP3s, and serves a web UI plus a JSON API consumed by official iOS and Android apps.

Why an audiobook server and not a music server: music servers (Navidrome, Plex, Jellyfin) track “what song you played last.” Audiobook servers track “second 247 of track 14 of book X” and sync that position across every device. For anything where you pause mid-sentence and resume tomorrow, like a language course or a long audiobook, that’s the whole point. So Audiobookshelf here is strictly for **spoken** audio: audiobooks, courses, and (later) podcasts. If I add music down the line it gets its own dedicated server, like **Navidrome**, in its own folder, not this one.

In my example the audio starts on CDs, so I need a CD/DVD drive somewhere. Use whatever you've got: your laptop or desktop's built-in drive, or a USB external plugged into the laptop or the Pi. It doesn't matter which, because the goal is the same either way, the audio ending up in `~/audiobookshelf/media/Audiobooks on the Pi`. If your audio is already files on disk, skip the ripping and just copy it into `~/audiobookshelf/media/Audiobooks` on the Pi (over the tailnet with `scp` or `rsync`, or off a USB stick).

Prerequisites

- You finished [Chapter 1](#): the Pi (`homelab`) is on your tailnet with Docker installed, and your laptop and iPhone are on the tailnet too.
- A **CD/DVD drive with the disc inserted**, on whichever machine is handy: a built-in drive, or a USB external plugged into the laptop or the Pi. You'll do the ripping on that machine, then make sure the files land on the Pi.
- The Pi (`homelab`) reachable over SSH for the server steps.

On the machine that has the drive, check it's seen:

```
lsblk -o NAME,LABEL,FSTYPE,MOUNTPOINT | grep -i -E "rom|cdrom|sr"
```

You should see `sr0`. If it isn't mounted (a headless Pi won't auto-mount it), mount it yourself; this prints the mount point (something like `/media/you/<LABEL>`):

```
udisksctl mount -b /dev/sr0
```

Part A: Rip and copy the audio

Step 1: Inspect what's actually on the disc

Don't assume the layout. List the mount point (substitute the real label that `udisksctl` just printed):

```
ls /media/$USER/<disc-label>/
```

A language-course data CD like mine typically has three useful things:

1. A **folder of full lessons** with 100 files named `L001-LESSON.mp3 ... L100-LESSON.mp3`: one complete lesson per file, pre-assembled in playback order.
2. **Per-lesson folders** (`L001-... ... L100-...`) that split each lesson into tiny per-sentence files. You can ignore these; the full lessons above are all you need.
3. A **User's Manual** in HTML and PDF.

Plus an `autorun.inf` (a Windows launcher you can ignore on Linux). It's a **data CD**, so this is the fast path: a plain file copy, no audio-CD ripping or re-encoding.

Step 2: Use the one-file-per-lesson folder

Copy the folder with **one complete lesson per file**. That way Audiobookshelf treats the course as a single audiobook with 100 sequential chapters: tap “next chapter” to advance one lesson, and your resume position lives at the lesson level, which is the right unit of progress for a language course.

Step 3: Get the audio onto the Pi

Because it’s a data CD, “ripping” is just copying files off the mounted disc. The audiobook library lives at `~/audiobookshelf/media/Audiobooks` on the Pi (we keep the content tidy inside the Audiobookshelf folder, set up in Step 4). Do the copy on whichever machine has the drive.

3a. Copy the lessons

If the drive is on the Pi, copy straight into the library:

```
mkdir -p ~/audiobookshelf/media/Audiobooks/"My Course"
rsync -av --no-perms --progress \
    "/media/$USER/<disc-label>/<lessons-folder>/" \
    ~/audiobookshelf/media/Audiobooks/"My Course"/
```

`rsync` over `cp` because if the disc has a read error or you Ctrl-C halfway, re-running resumes where it stopped. `--no-perms` matters: a CD is mounted read-only (files at `r-----`), and the `-a` archive flag would copy those modes to the destination, leaving files you can barely read and directories you can’t write into. `--no-perms` uses your `umask` defaults instead, so the result behaves like normal files you created. The L001 ... L100 names are already zero-padded, so Audiobookshelf orders them correctly.

3b. If the drive is on your laptop

Rip into any folder on the laptop, then push it to the library path on the Pi over the tailnet (works from anywhere, fastest on the same LAN):

```
rsync -av --no-perms --progress \
    "/media/$USER/<disc-label>/<lessons-folder>/" ~/ "My Course"/
rsync -avz --partial --no-perms --progress \
    ~/ "My Course"/ you@homelab:~/audiobookshelf/media/Audiobooks/"My Course"/
```

`-z` compresses over the wire and `--partial` keeps partial files so an interrupted transfer resumes quickly, useful for a GB-scale copy.

3c. Optional: save the manual

To keep the disc's User's Manual alongside the lessons, copy its folder into `~/audiobookshelf/media/Audiobooks/Course/_manual/` the same way. The leading underscore sorts it to the bottom so Audiobookshelf scans the lessons first.

3d. Unmount and eject

When the copy is done, unmount and eject on the machine that had the drive:

```
udisksctl unmount -b /dev/sr0
eject /dev/sr0
```

You only do this once; from now on the files live on the Pi's disk.

Part B: Run the server

Step 4: Run Audiobookshelf on the Pi

SSH into the Pi if you aren't already there (`ssh you@homelab`); Docker is installed from Chapter 1. We'll run Audiobookshelf as a small **Docker Compose** project, one folder with one `compose.yaml`, the same self-contained pattern every other service in this series uses (so it's one lifecycle to learn and a single file to back up).

```
mkdir -p ~/audiobookshelf/config ~/audiobookshelf/metadata #
media/Audiobooks already exists from Step 3
cd ~/audiobookshelf

cat > compose.yaml <<'EOF'
services:
  audiobookshelf:
    container_name: audiobookshelf
    image: ghcr.io/advplyr/audiobookshelf:latest

    ports:
      - "13378:80"          # host 13378 -> container 80 (host 80 stays free
for Caddy)

    volumes:
      - ./media/Audiobooks:/audiobooks    # add ./media/Podcasts:/podcasts
here later
      - ./config:/config
      - ./metadata:/metadata

    restart: unless-stopped
EOF
```

Then keep the runtime data out of version control and launch:

```
printf 'config/\nmetadata/\nmedia/\n' > .gitignore
docker compose up -d
```

i Why everything lives under ~/audiobookshelf/

The whole stack is **self-contained** in one folder: the `compose.yaml`, the app's `config/` and `metadata/`, and the content itself under `media/`. Because all the volume paths are relative (`./media/Audiobooks`, `./config`), you can move or back up the entire homelab service by copying that one directory. When you add a **Podcasts** library later, it's just another folder, `~/audiobookshelf/media/Podcasts`, mounted at `/podcasts`. (Audiobookshelf is for *spoken* audio; music gets its own server, see the note below.)

What the key settings do:

- **restart:** `unless-stopped` auto-starts the container on boot and after crashes, exactly what you want on an always-on Pi.
- **ports:** `13378:80` maps host port 13378 to the container's port 80 (we avoid host port 80 to leave it free for the reverse proxy in Chapter 4).
- The **volumes** expose your audio under `media/` to the container and persist the database (listening positions, users, settings) next to the compose file, so recreating the container loses nothing. The paths are relative to `~/audiobookshelf/`.

Verify it's running:

```
docker compose ps
curl -I http://localhost:13378      # expect HTTP/1.1 200 OK
```

Part C: Connect and use it

Step 5: Initial setup in the browser

From any browser on your tailnet, open `http://homelab:13378`. The first-run wizard creates the root user, then drops you at an empty dashboard.

5a. Create the root user

This is your admin account. Pick a strong password, Audiobookshelf has no password-reset flow; if you forget it, recovery means editing the SQLite database in `/config` by hand.

5b. Add the primary library

Settings (gear) → Libraries → Add Library:

- **Media type: Books.** (This changes the *data model*: Books track one resume position per book in seconds; Podcasts track per-episode played booleans. You want the first for language lessons.)
- **Library name:** *My Course* (specific) or *Audiobooks* (ages better if you'll add more courses). Pick one style and stick to it.
- **Metadata providers:** leave **Open Library** enabled. Even for a homemade rip of a published course it found a clean match for me (cover art and title), where the audiobook-store providers mostly draw blanks on non-commercial audio. You can uncheck the rest to keep scans quick. (For real published audiobooks later, Audible and Google Books are good additions.)
- **Folder:** browse to `/audiobooks`, the *container* path, not the host path. Inside Docker, `~/audiobookshelf/media/Audiobooks` is mounted at `/audiobooks` (from the `volumes:` mapping in the `compose.yaml` in Step 4). If you see `/home/you/Audiobooks` in the picker instead, something's wrong with the volume mount.

Save. Audiobookshelf scans and creates one audiobook with 100 chapters (L001-LESSON.mp3 ... L100-LESSON.mp3) in about 30 seconds.

5c. Verify the scan

Open the library, open *My Course*: you should see 100 numbered chapters. Click chapter 1. It should play in the browser. If it does, the server is fully functional.

Step 6: Connect the iPhone

1. Install **Audiobookshelf** from the App Store (by `advplyr`, same as the server). Tailscale is already installed and signed in from Chapter 1.
2. Open it, tap **Add Server**:
 - **Server address:** `http://homelab:13378` (or `http://100.x.y.z:13378` using the Pi's Tailscale IP).
 - **Username / password:** the root user from Step 5a.
3. Tap your library → your audiobook → the first track. Audio should stream within a second or two, on Wi-Fi or cellular, anywhere.

Step 7: Daily use

- **Playback speed:** 0.5×–3.0×. 0.85× is great for shadowing dialogue you don't fully understand; 1.25× for review passes. (Enable "Remember playback speed per book" in iOS settings, or it resets to 1.0× each track.)
- **Skip & scrub:** ±10s / ±30s buttons; tap-and-hold to scrub.
- **Bookmarks:** drop a named marker at the current position for phrases to revisit.
- **Cross-device progress:** pause on the iPhone, open the web UI at home, resume at the same second.
- **Offline downloads:** tap the download icon to cache a book locally, your saving grace if a network is hostile to VPNs. Downloaded audio plays with no network at all.

Maintenance

Update Audiobookshelf (on the Pi):

```
cd ~/audiobookshelf
docker compose pull && docker compose up -d
```

Because config and metadata live in `~/audiobookshelf/`, recreating the container preserves everything.

Back up listening progress with a cron job on the Pi:

```
0 3 * * * tar czf ~/backups/abs-$(date +%F).tar.gz ~/audiobookshelf/config
~/audiobookshelf/metadata
```

Add more material: drop another folder under `~/audiobookshelf/media/Audiobooks/` (e.g. a Pimsleur course) and click “Scan Library.” Each top-level folder becomes its own audiobook.

Troubleshooting

The iPhone app says “Cannot connect.” From the phone’s Safari, visit `http://homelab:13378`. If Safari also fails, Tailscale isn’t up, open the Tailscale app and confirm both devices show green dots. If Safari works but the app fails, you typed the wrong port (13378).

Tracks play in the wrong order. Audiobookshelf sorts by filename. Rename so they sort correctly (Lesson 01.mp3, not Lesson 1.mp3), then “Re-scan” in the web UI.

Container won’t restart after a reboot. `cd ~/audiobookshelf && docker compose ps`; if it’s down, `docker compose up -d`. If `restart: unless-stopped` isn’t taking effect, check `systemctl status docker`, Docker itself may not be enabled at boot.

rsync reports “Permission denied” on every mkdir. Check the destination mode: `ls -ld /path`. A `dr-x-----` directory is missing its write bit; fix with `chmod u+w /path` (or `chmod 755`) and re-run.

Work Wi-Fi blocks Tailscale. Tailscale falls back to DERP relays automatically (slower but works). Last resort: download lessons to the iPhone before leaving home, cached audio plays fully offline.

Recap

- **Copied** the audio off the disc straight onto the Pi with `rsync --no-perms`.
- **Ran** Audiobookshelf on the Pi as a small `docker compose` project.
- **Connected** the iPhone app to `http://homelab:13378`.

Your course is now permanently available from any device on your tailnet, with proper progress tracking and no subscription. Next, in [Chapter 3](#), we give the Pi a second job: blocking ads for every device on your home network with Pi-hole.